

Reproduction Report of Deep Directly-Trained Spiking Neural Networks for Object Detection

Senquan Zhang
Alexandru Eugen Bulboaca (5483115)
Adhiraj Kaushik (6396364)

1 Introduction

Most of the object detection models we use today, namely the YOLO family, the RCNN family, transformer-based detectors, etc. are built on artificial neural networks (ANNs). They're accurate, but they're also energy expensive: every layer involves dense multiply-accumulate (MAC) operations on floating-point numbers, and running these models at scale or on embedded/edge hardware comes with a real energy cost. Spiking Neural Networks (SNNs) are often pitched as a more biologically plausible and energy-efficient alternative, and over the past few years there's been growing interest in whether they can match ANN performance while using a fraction of the power.

The core idea behind an SNN is simple but quite different from standard ANN. Rather than propagating continuous activation values through the network, a spiking neuron accumulates input over time as a membrane potential, and only fires a discrete, binary spike once that potential crosses a threshold. The most commonly used neuron model for this is the Leaky Integrate-and-Fire (LIF) neuron, which adds a decay term so that, like a biological neuron, unused charge "leaks away" over time rather than accumulating indefinitely. Because neurons only communicate when they spike, and because spikes are binary, the network's computation becomes sparse and event-driven: a neuron that never fires costs (almost) nothing. On neuromorphic hardware, where this sparsity can be exploited directly, this translates into substantial energy savings over dense ANN inference.

Historically, there have been two main ways to get an SNN: convert a pre-trained ANN into a spiking equivalent (ANN-SNN conversion), or train the spiking network directly using a surrogate gradient to work around the fact that the spike function itself isn't differentiable. Conversion methods are simpler to implement since they reuse existing ANN weights, however, to approximate the ANN's continuous activations well enough, they typically need very long inference windows, about 100s - 1000s of time steps; which erodes most of the efficiency gains SNNs are supposed to offer. They also don't transfer well to event-based camera data, since they're fundamentally trying to mimic a static ANN activation rather than capture genuine spatio-temporal dynamics. Directly-trained SNNs avoid both problems: they can reach strong performance in only a handful of time steps, and they're naturally suited to processing event streams.

Direct training has already been shown to work well for image classification, where deep spiking ResNet variants have pushed network depth past 100 layers without running into the vanishing/exploding gradient problems that plagued earlier attempts. Object detection is different. It's a regression task, not just a classification one, the network has to localize multiple objects of varying scale and produce continuous bounding-box coordinates, which demands a deeper and more representationally rich backbone than classification typically needs. Prior SNN-based detectors have mostly sidestepped this difficulty by either relying on ANN-SNN conversion (with

the long-time-step penalty described above) or by using a hybrid setup, a spiking backbone bolted to an ANN detection head which reintroduces the energy and parameter costs SNNs are meant to avoid.

This is the gap that the paper we reproduce from the paper: Deep Directly-Trained Spiking Neural Networks for Object Detection (Su et al., ICCV 2023) [1]. The authors propose EMS-YOLO, which they describe as the first attempt to train a deep SNN end-to-end with surrogate gradients for object detection, without converting from an ANN and without falling back on non-spiking components in the detection head. The rest of this report walks through their method, our reproduction of it, the changes we had to make to get their code running in our environment, and the results we were able to obtain on COCO2017 and an event-based dataset.

2 Background

2.1 The problem with existing residual structures

The central design problem the paper addresses is energy efficiency in deep spiking networks, specifically in the residual blocks that make depth possible in the first place. Two existing approaches: SEW-ResNet [2] and MS-ResNet [3] had already extended directly-trained SNNs past 100 layers for classification, but both have a structural weakness once you try to use them for detection.

In SEW-ResNet [2], both the residual path and the shortcut path end in a spiking activation, and the two are summed. The problem is that when both paths spike at the same time step, the sum can equal 2, which is no longer a binary spike. Feeding that into the next block's convolution turns it back into a non-spike (MAC) operation. There are ways to suppress this (AND or IAND operations instead of addition), but the paper notes these come at a real cost to performance.

MS-ResNet avoids that particular issue by keeping the shortcut path in the membrane-potential domain rather than the spike domain, only converting to spikes at certain points. But this means that whenever the shortcut path needs to change the number of channels or downsample, which happens constantly in objection detection, since object detection requires extracting features at multiple scales, it has to use a non-spike convolution to do so. In a classification network these channel/dimension changes are relatively infrequent, so the cost is tolerable; in a detection network, where multi-scale feature extraction is the norm, the authors argue this becomes a real source of energy consumption.

2.2 EMS-ResNet

The paper's answer is EMS-ResNet (Energy-efficient Membrane-Shortcut ResNet), a residual block designed so that every path including the shortcut path only carries spikes and not raw membrane potentials or non-spike activation. The residual path follows the same general approach as MS-ResNet [3]. The key change is on the shortcut path: whenever the number of channels needs to change, the authors insert an additional LIF neuron before the convolution, so the convolution always operates on sparse binary spikes rather than dense values.

Because a single design doesn't elegantly handle both "channels staying the same or shrinking" and "channels increasing," the paper splits this into two block variants:

- EMS-Block1: Used when the channel count is constant or decreasing.
- EMS-Block2: Used when the channel count increases. Rather than a strided convolution, this version uses a max-pool plus a concatenation operation to reuse features while keeping the number of new parameters low.

Stacked together, these blocks form the EMS-ResNet backbone, with the depth and channel width adjustable depending on the task. The paper reports results at depths of 10, 18, and 34 layers (EMS-Res10/18/34).

2.3 Why this doesn’t just vanish or explode?

A natural problem with any sufficiently deep network whether it is spiking or not is if gradients will vanish or explode during training. The paper addresses this using Gradient Norm Equality (GNE) theory, treating each block in the network as a transformation with its own Jacobian and analyzing the expected normalized trace of that Jacobian across blocks.

The authors of the paper give a full proof and the practical claim is this: as long as each block in the network keeps this quantity close to 1, gradients propagate through arbitrarily deep stacks of blocks without systematically shrinking or blowing up. The authors show that pooling, upsampling, and concatenation operations all satisfy the conditions needed for this analysis, and that EMS-ResNet specifically can be tuned (via the second moment of its input) to satisfy the $\simeq 1$ condition, giving a theoretical basis for why the architecture should support deep training, not just an empirical one.

2.4 The full EMS-YOLO model

EMS-YOLO wraps the EMS-ResNet backbone in a YOLO-style detection framework. Static images are simply copied across each of the T time steps as input (a standard approach for applying SNNs to non-temporal data); event-camera data is instead split into temporal bins, with each bin treated as one time step’s input. The backbone extracts multi-scale features through a stack of EMS-Blocks, and the detection head itself is also rebuilt out of EMS-Blocks rather than the conventional stack of directly-connected convolutions, so that the entire network, backbone and head, remains full-spike. The final bounding box and class predictions are read out from the membrane potential of the last layer’s neurons (rather than from spikes directly), since that potential is continuous and well-suited to regressing real-valued coordinates.

2.5 Headline results

The paper validates EMS-YOLO on two very different kinds of data: the frame-based COCO2017 dataset and the event-based Gen1 Automotive Detection dataset. On COCO, their headline comparison is against ANN-SNN conversion methods such as, Spiking-YOLO, Bayesian-Optimization-based conversion, and Spike Calibration, all of which require anywhere from 64 to several thousand time steps to reach competitive accuracy. EMS-YOLO, trained directly, reaches a higher mAP@0.5 than all of them using only 4 time steps.

On Gen1, EMS-Res10 outperforms a same-sized ANN-ResNet10 baseline and a prior SNN-based detector built on SEW-ResNet, while maintaining a low spike firing rate (around 21%), which the authors translate into an energy-consumption estimate roughly 4.9-5.83x lower than the equivalent ANN, depending on the comparison setting. They also run ablations on residual block type, network depth, and number of time steps, the latter being the one most relevant to the part of the paper we focus on reproducing.

3 Methodology

3.1 Models, Datasets and Metrics

To establish a clear experimental framework for our replication, we define the primary model architectures, datasets, and evaluation metrics used throughout our experiments.

3.1.1 Models

The focal point of our reproduction is the **EMS-YOLO** model proposed in the paper [1]. It is a YOLO-style object detector built entirely on Spiking Neural Networks (SNNs) using surrogate gradient direct training. Unlike conventional SNN detectors that rely on non-spiking layers in the detection head, EMS-YOLO retains a full-spike pipeline from feature extraction to bounding box regressions by leveraging the **Energy-efficient Membrane-Shortcut ResNet (EMS-ResNet)** backbone. This backbone employs two residual block configurations: EMS-Block1 (for constant or decreasing channels) and EMS-Block2 (for increasing channels, using max pooling and concatenation to preserve spiking sparsity). We focus our baseline and ablation studies on the **EMS-Res10** variant (a 10-layer spiking backbone). For baseline comparisons, we refer to the standard non-spiking **ANN-ResNet10** detector, which operates on conventional floating-point multiply-accumulate (MAC) operations.

3.1.2 Datasets

We utilize three primary datasets to validate static image representation, temporal scaling, and control properties:

- **COCO2017 Dataset:** The Microsoft Common Objects in Context (COCO) dataset is the gold standard for static frame-based object detection, containing over 118,000 training images and 5,000 validation images across 80 categories. In our validation run, static images are repeated across T steps to stimulate the SNN.
- **Prophesee Gen1 Automotive Dataset:** A large-scale event-based dataset recorded using an ATIS event sensor with a spatial resolution of 304×240 pixels. It contains over 39 hours of driving footage in urban, highway, and rural environments, featuring labels for cars and pedestrians. It serves as our primary event-based object detection benchmark.
- **Prophesee N-Cars Dataset:** A binary event-based classification dataset containing 15,422 sequences of 100ms event camera captures, categorized into cars and background classes. We employ N-Cars as a simplified control dataset to isolate the effects of temporal steps T from localization noise.

3.1.3 Metrics

The performance of the models is quantified using standard object detection and SNN-specific efficiency metrics:

- **Mean Average Precision (mAP):** The principal metric for evaluating detection performance:
 - **mAP@0.5:** Evaluates the average precision at a single Intersection-over-Union (IoU) overlap threshold of 0.5, measuring general detection and classification performance.
 - **mAP@0.5:0.95:** Evaluates the average precision across ten IoU thresholds ranging from 0.5 to 0.95 with a step of 0.05, placing a high premium on precise bounding-box localization.
- **Classification Accuracy:** Applied specifically to the N-Cars control dataset run, evaluating the proportion of correctly classified sequences.
- **Spike Firing Rate:** The average percentage of neurons that fire a binary spike per time step. This metric represents network sparsity, which directly determines SNN energy consumption since computation is only triggered by spikes.

3.2 Experiments

To verify the core claims and scaling properties of EMS-YOLO, we structured our reproduction into five sequential experimental stages. These runs were designed to isolate hyperparameter sensitivity, assess temporal resolution scaling, and verify generalisability across distinct datasets:

1. **Baseline Reproduction (Gen1 Dataset):** We trained the EMS-ResNet10 detector on the event-based Gen1 Automotive dataset under the authors’ default baseline configuration ($T = 5$ time steps, batch size of 16, and learning rate of 0.01) for 50 epochs to verify that a directly-trained SNN can achieve the reported object detection metrics.
2. **COCO2017 Dataset Validation:** Because training the full frame-based COCO2017 dataset from scratch is computationally prohibitive in a resource-limited academic allocation, we performed validation on the test split using the pre-trained weights provided by the authors. This verified the model’s compatibility with static image formats (where frames are duplicated T times).
3. **Hyperparameter sensitivity sweep:** We swept model parameters across two dimensions to verify the optimization stability of the spiking layers:
 - **Batch Size:** Swept across $BS \in \{8, 16, 32, 64\}$ at a constant learning rate ($LR = 0.01$) to evaluate how stochastic gradient noise affects firing threshold stability and final mAP metrics.
 - **Learning Rate:** Swept across $LR \in \{0.001, 0.1\}$ at a constant batch size ($BS = 16$) to evaluate the convergence boundaries.
4. **Temporal Step Ablation Study:** The core temporal resolution hypothesis states that performance scales monotonically with the temporal step parameter T . We performed training sweeps across $T \in \{1, 2, 4, 6\}$ (extending the paper’s original set of $\{1, 3, 5\}$) to study the trade-off between temporal resolution and active spike density.
5. **N-Cars Control Dataset Run:** We trained the model on the N-Cars classification dataset, mapped to a detection format using full-frame bounding boxes $([0.5, 0.5, 1.0, 1.0])$, to evaluate the model’s capacity to serve as a pure sequence classifier and generalise to different event-based camera data.

3.3 DelftBlue Cluster Workspace

Executing these compute-heavy SNN workloads on DelftBlue introduced several cluster-specific constraints and scheduling challenges that required a specialized deployment architecture:

- **Workspace and Quota Management:** DelftBlue enforces a strict **30 GB** home quota ($/home/NetID$), which is easily exceeded by PyTorch caches, raw datasets, and checkpoint outputs. To circumvent this, the entire repository was migrated to the **5 TB** scratch file system ($/scratch/NetID$), and a symbolic link was created from the home directory to ensure all writes directed to scratch.
- **Synchronization Pipeline:** Development was maintained locally, and code changes were pushed to the cluster via an anchored synchronization script (see Appendix A.1). The Weight & Biases log directory was anchored in the exclude configuration ($--exclude = /wandb/$) to prevent namespace collisions and that some non-interactive batch jobs timing out waiting for Weights & Biases credentials.
- **Slurm Memory Allocations:** To support the RAM caching algorithm detailed below, the Slurm configuration requested 16 CPUs per task and 8000 MB per CPU ($--cpus-per-task = 16$ and $--mem-per-cpu = 8000M$), providing a total memory allocation

of 128 GB. This satisfied the cluster memory boundaries while providing enough RAM to cache the event dataset (an example job script configuration is detailed in Appendix A.2).

- **GPU Partition Migration (Compute Capability Mismatch):** Default GPU partition scheduling assigns Tesla V100 cards (Compute Capability 7.0). However, the pre-installed PyTorch build on DelftBlue (+cu130) compiles kernels specifically for Compute Capability ≥ 7.5 (Ampere and later architectures), causing immediate `AcceleratorError` crashes. We resolved this by modifying all Slurm configurations to explicitly target A100 GPU nodes (`--partition = gpu - a100`), which support Compute Capability 8.0.
- **Environment Mismatches and Deprecations:** We resolved version differences on the compute nodes, including replacing deprecated NumPy type casts and wrapping Pillow font calls with text bounding box fallbacks (see Appendix B.6) to prevent crashes in visual plotting threads. We also replaced hardcoded `cuda:0` device assignments with PyTorch’s dynamic `select_device(opt.device)` to allow CPU dry-runs on login nodes.

4 EMS-YOLO Adaptations

The starting point for our implementation was the official repository provided by the authors: BICLab EMS-YOLO. While designed for local workstations with direct hardware access and legacy library dependencies, running this codebase at scale on the DelftBlue cluster required reconciling compatibility mismatches, cross-modal dataloading interfaces, GPFS file system bottlenecks, and memory limitations.

4.1 Dataloader Alignment for Frame and Event Modalities

To evaluate the model’s performance on both frame-based (COCO2017) and event-based (Gen1, N-Cars) datasets using a unified evaluation engine, we had to reconcile fundamental structural mismatches at the data-model interface.

Spiking Neural Network (SNN) layers process temporal step inputs and require 5D tensors of shape $(batch, T, channel, H, W)$. The static COCO dataloader, however, yields 4D image tensors $(batch, 3, H, W)$ and returns a 4-element tuple (containing image shapes), whereas the event dataloader yields 3-element tuples. This discrepancy caused shape unpacking and dimension crashes during model warmup and inference.

We resolved these mismatches by implementing a dynamic tuple length check in the validation loop to conditionally unpack the image metadata (see Appendix B.1). To feed static images into the spiking backbone, we developed an input adapter that replicates the 4D image tensor T times along a new temporal dimension (see Appendix B.2). Finally, we patched the model warmup routine to initialize a 5D dummy tensor, preventing dimension mismatch errors at startup.

4.2 High-Performance Dataset Preprocessing

Preparing the asynchronous event streams for SNN execution represented a major preprocessing milestone.

Gen1 H5 Extraction Optimization: The raw Gen1 dataset is stored in massive, sequential ‘.h5’ files. Slicing these files on the fly during training is extremely I/O intensive. On DelftBlue’s parallel GPFS file system, querying the H5 file multiple times per bounding box timestamp resulted in an average preprocessing time of 49 seconds per sequence. This severe I/O bottleneck caused our Slurm preprocessing jobs to time out under the 6-hour limit. We resolved this by loading the entire sequence events array into memory once at the start of each file’s processing (see Appendix B.4). This minimized random disk lookups, reducing sequence processing

times from 49 seconds to 1-2 seconds (a 25x–50x speedup) and allowing dataset preparation to complete.

N-Cars Classification-to-Detection Mapping: To evaluate the temporal resolution hypothesis, we mapped the binary classification N-Cars dataset into a YOLO-compatible detection format without modifying the EMS-YOLO architecture. For positive samples (cars), we generated a virtual, full-frame bounding box centered at (0.5, 0.5) with scale (1.0, 1.0) spanning the entire frame. For background samples, we generated empty label text files (see Appendix B.5). This successfully framed the classification task as a binary detection task, allowing us to evaluate the SNN feature extractor’s representation quality under various T steps.

4.3 Shared Memory Optimisation and Parallel RAM Caching

Training on the preprocessed Gen1 dataset requires loading 72481 ‘.npy’ event frame arrays per epoch. Running this directly from scratch disk storage caused GPFS parallel file system bottlenecks, making a single training epoch take 45 minutes on an A100 GPU and causing Slurm jobs to exceed the 4-hour limit.

The Multiprocessing Memory Duplication Bug: To bypass GPFS disk reads, we enabled PyTorch’s in-memory RAM caching (`--cache`), which pre-loads frames into memory on the first epoch. However, because PyTorch dataloaders spin up multiple worker processes (`num_workers = 4`), and caching was handled dynamically during the dataset’s `__getitem__` execution inside these child worker processes, each worker created its own separate copy of the 111 GB cache. This namespace isolation raised memory usage to 444 GB, triggering cgroup OOM-killer crashes.

Pre-fork Parallel Caching Solution: We redesigned the dataloader class to cache the entire dataset in the parent process during initialization (`__init__`), before worker processes are forked. To prevent slow sequential startup times, we implemented parallel loading using a thread pool (see Appendix B.3). When PyTorch forks the worker processes, they inherit the parent’s pre-populated cache. Since the workers only read from this cache, Linux’s Copy-On-Write (COW) memory sharing allows them to share the exact same physical RAM space. This eliminated all cache duplication, maintaining peak RAM usage below 120 GB (within our 128 GB Slurm allocation) and reducing epoch times to 2 minutes (a 20x speedup).

5 Results

The following section presents the results of our EMS-YOLO reproduction according to the five experimental stages defined in the methodology. Rather than reporting only final accuracy numbers, each experiment is used to answer a specific reproduction question. Because several experiments were constrained by some limitations which will be discussed in detail in next section, the reported results are interpreted as an empirical reproduction study rather than a complete re-training of every setting in the original paper.

5.1 Baseline Reproduction (Gen1 Dataset)

This experiment aims to verify that the Gen1 baseline can be reproduced under our DelftBlue setup. The paper claimed that the model can achieve the mAP@0.5:0.95 at 0.267, mAP@0.5 at 0.547 under $T=5$ temporal steps.

Table 1: Gen1 Dataset: Paper vs. Reproduced Baseline (EMS-Res10, T=5)

Source	Epochs	Params	Firing Rate	mAP@0.5:0.95	mAP@0.5
Paper [1]	50	6.20M	21.15%	0.267	0.547
Our Reproduction	33	6.20M	–	0.343	0.623

Our Gen1 baseline experiment was conducted with the EMS-ResNet10 variant using T=5 temporal steps. The run reached its best validation result at epoch 17, with mAP@0.5 of 0.623, and mAP@0.5:0.95 of 0.343. Compared with the EMS-ResNet10 Gen1 result reported in the original paper, our reproduced run achieved higher detection performance on both metrics.

The mAP@0.5 increased rapidly during the first few epochs and became stable after approximately epoch 11 as seen in Fig. 1. The best mAP@0.5 was reached at epoch 17, while later epochs fluctuated within a narrow range without producing additional improvement.

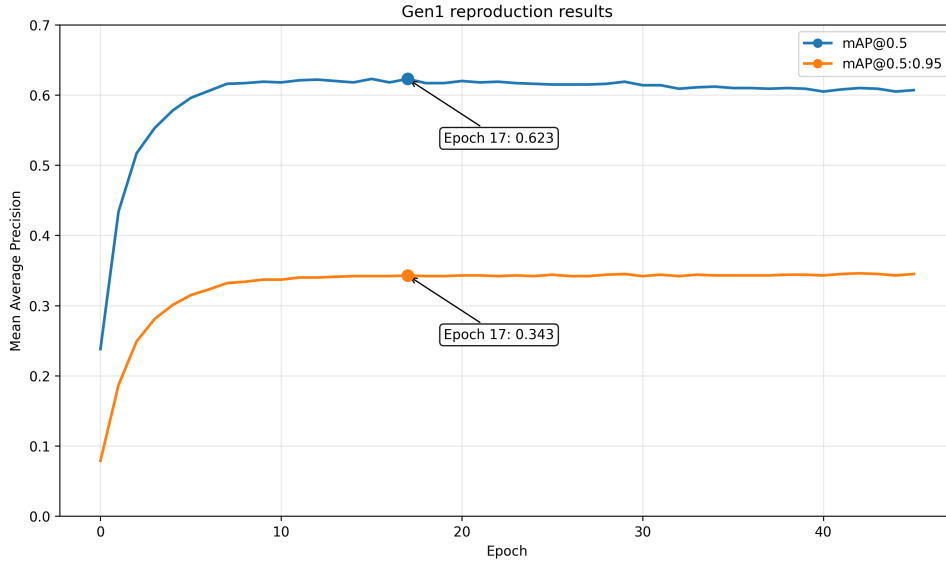


Figure 1: Reproduction results of the Gen1

Overall, the Gen1 baseline was successfully reproduced under our DelftBlue setup in terms of object detection accuracy. Our reproduced model not only matched the reported EMS-ResNet10 baseline, but also achieved higher mAP values on both evaluated detection metrics.

5.2 COCO2017 Dataset Validation

This experiment aims to verify the model’s compatibility with static image formats. Different from the Gen1 baseline experiment, this experiment was not a full training reproduction from scratch. Instead, we used pre-trained weights provided by the authors and tested whether our modified EMS-YOLO could correctly process the COCO2017 validation dataset.

In the original paper, the EMS-YOLO model with EMS-ResNet34 is reported to achieve an mAP@0.5 of 0.501 on COCO2017 using T=4 temporal steps.

Table 2: COCO2017 Dataset: Paper vs. Reproduced Results (EMS-ResNet34, T=4)

Source	Model	mAP@0.5
Paper [1]	EMS-ResNet34	0.501
Reproduction	EMS-ResNet34	0.508

The reproduced model achieved mAP@0.5 of **0.508**, and mAP@0.5:0.95 of **0.308**. The reproduced mAP@0.5 is slightly higher than the value reported in the original paper. This indicates that the released weights, dataset loading procedure, and static-image-to-temporal-input adaptation were functioning correctly in our environment.

The class-wise results also show a reasonable pattern for COCO object detection. Large and visually distinctive categories such as airplane, train, elephant, zebra and giraffe achieved higher mAP@0.5 values, while smaller categories such as backpack, book, and knife achieved lower metric values.

Table 3: Representative class-wise COCO2017 validation results.

Class	Images	Labels	P	R	mAP@0.5	mAP@0.5:0.95
all	5000	36335	0.662	0.460	0.508	0.308
airplane	5000	143	0.829	0.755	0.826	0.549
train	5000	190	0.920	0.742	0.842	0.586
elephant	5000	252	0.717	0.817	0.801	0.529
zebra	5000	266	0.856	0.801	0.883	0.589
giraffe	5000	232	0.868	0.832	0.890	0.631
backpack	5000	371	0.443	0.188	0.190	0.0766
book	5000	1129	0.443	0.122	0.159	0.0581
knife	5000	325	0.492	0.120	0.149	0.0696

Overall, the COCO2017 validation experiment successfully validated our model with released EMS-YOLO weights have the compatibility with static image formats. However due to limitations suggested in Section 3.2, a complete training reproduction with 300 epochs couldn’t be performed. Despite reproducing for training, the results on the testing dataset results validate the claims on the COCO 2017 Dataset performance of the paper as seen in fig. 6.

5.3 Hyperparameter sensitivity sweep

This experiment aims to analyze sensitivity of the directly-trained SNN detector is to batch size and learning rate. Unlike the Gen1 and COCO2017 experiments, the purpose of this experiment is not to reproduce the results, but to test whether the training behavior of EMS-YOLO is stable under reasonable optimization of hyperparameters.

First, we tested the effect of batch size while keeping the learning rate fixed at 0.01 and the temporal step fixed at T=5. The tested batch sizes were 8, 16, 32, and 64. As shown in Table 4, all four settings reached similar mAP values, which indicates that the model is relatively robust to batch size in this range. However, the best performance was obtained with batch size 8, reaching an mAP@0.5 of 0.626 and an mAP@0.5:0.95 of 0.345. Increasing the batch size did not improve the performance; instead, the peak mAP@0.5 decreased slightly from 0.626 at batch size 8 to 0.620 at batch size 64.

Table 4: Batch-size sensitivity results on the Gen1 dataset with T=5 and learning rate 0.01.

Batch Size	Best Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95	GPU Memory (GB)
8	17	0.732	0.581	0.626	0.345	6.92
16	10	0.715	0.578	0.623	0.339	13.2
32	20	0.734	0.582	0.622	0.342	25.8
64	13	0.716	0.576	0.620	0.340	50.7

The batch-size sweep shows that larger batch sizes do not provide a significant accuracy advantage for this EMS-YOLO setup. Instead, smaller batch sizes slightly improve the validation mAP while requiring less GPU memory. Since the differences in mAP are small, batch size appears to be a secondary hyperparameter for accuracy, but it remains important for computational feasibility. In practice, larger batch sizes will increase the GPU memory footprint and therefore the limitation will come from hardware resources rather than by model performance.

Second, we tested the effect of the learning rate while keeping the batch size fixed at 16 and T=5. The default learning rate used in our baseline configuration is 0.01. As shown in Fig. 2, lowering the learning rate to 0.001 resulted in slower convergence and worse performance. The best mAP@0.5 was only 0.567, suggesting that the model underfits within the available training budget. Meanwhile, increasing the learning rate to 0.1 produced fast early improvement and reached a best mAP@0.5 of 0.591 at epoch 6, but the validation performance became unstable afterwards and dropped substantially in later epochs. The default learning rate of 0.01 achieved the best and most stable result among the tested settings.

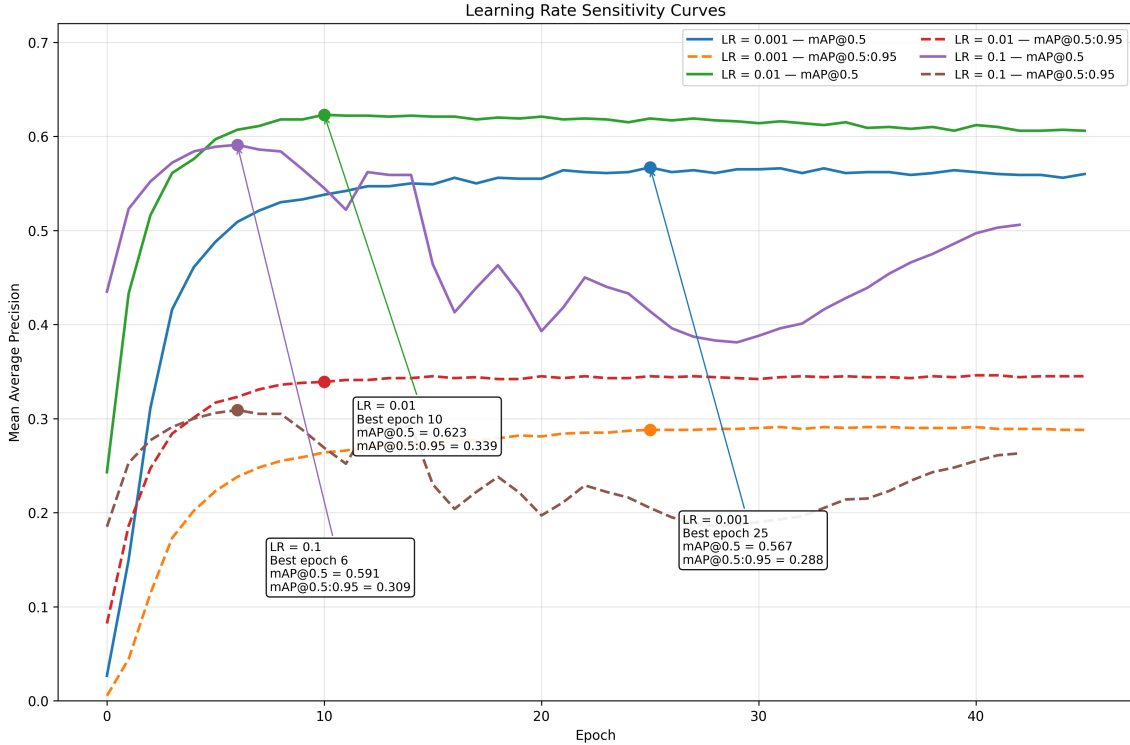


Figure 2: learning-rate sweep results

The learning-rate sweep shows that EMS-YOLO is more sensitive to learning rate than to batch

size. A learning rate that is too small leads to a stable but suboptimal plateau, while a learning rate that is too large causes unstable validation behaviour after an early peak. For a directly-trained SNN, the optimization is important since it must update both synaptic weights and threshold-based spiking dynamics. If the update step is too small, the model may not move efficiently through the firing-threshold regime; if it is too large, the learned spiking states can become unstable.

Overall, the hyperparameter sweep experiment shows that the default learning rate of 0.01 is a well-balanced choice for the EMS-YOLO training setup. Batch size mainly affects memory usage and produces only small differences in validation accuracy, while learning rate directly affects convergence quality and training stability. Therefore, to conclude this experiment, LR=0.01 and a small batch size such as 8 or 16 provide the best trade-off between accuracy, stability, and computational feasibility.

5.4 Temporal Step Ablation Study

This experiment aims to study the trade-off between temporal resolution and active spike density. In EMS-YOLO, the temporal dimension is important because the SNN integrates information through membrane potential dynamics across time steps. Therefore, increasing T should provide more temporal context, but it also increases the computational cost.

In the paper, the authors report the performance of EMS-ResNet10 based on the COCO2017 Dataset for $T=\{1,3,5,7\}$. As shown in Figure 7, this shows that increasing the number of time steps improves detection performance.

Table 5: Timestep Ablation: Paper $T = \{1,3,5,7\}$ [1] vs. Reproduction $T = \{1,2,4,6\}$

Paired T	T [1]	mAP@0.5	T (Reproduction)	mAP@0.5
Pair 1	1	0.328	1	0.569
Pair 2	3	0.362	2	0.588
Pair 3	5	0.367	4	0.607
Pair 4	7	0.383	6	0.625

In our reproduction, time step ablation was performed for $T = \{1,2,4,6\}$ as opposed to the authors $T = \{1,3,5,7\}$. This decision was made since the baseline reproduction for $T = 1$ was similar, hence applying intermediate time-steps could help find an in-between convergence point that the authors of the paper could’ve missed.

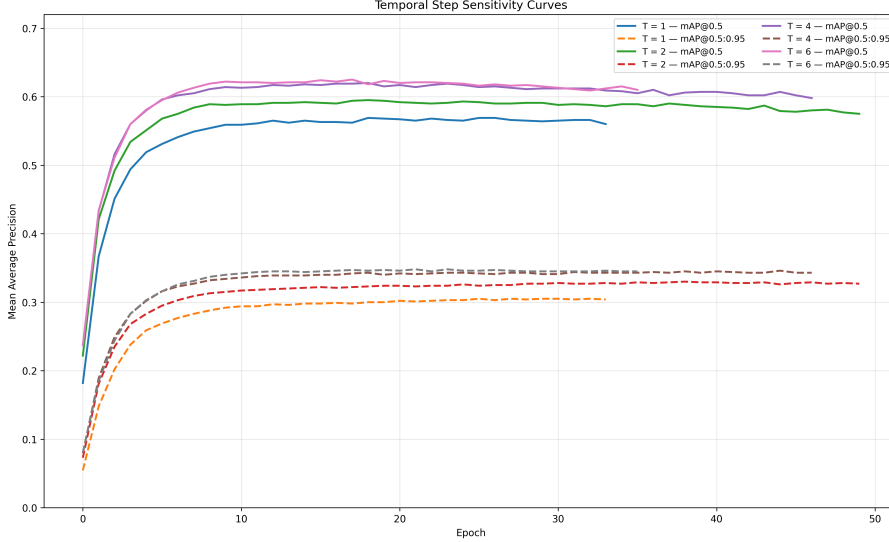


Figure 3: Temporal step ablation experiment results.

The ablation results show a clear improvement when the time step is increased. With $T=1$, the model reached only 0.569 mAP@0.5, which indicates that a single time bin is not sufficient to fully exploit the event-stream dynamics. Increasing T to 2 improved mAP@0.5 to 0.588, and increasing it further to 4 raised the score to 0.607. The $T=5$ baseline in Section 5.1 achieved mAP@0.5 of 0.623, while $T=6$ achieved the highest mAP@0.5 of 0.625.

These results support the same general trend as the original paper: larger time steps provide more temporal information and improve detection performance. However, the improvement becomes smaller once T reaches 4 or higher. The gain from $T=4$ to $T=6$ is only 0.005 in mAP@0.5, which shows the improvement from increasing T is at its limit. At the same time, increasing T also increases the computational cost because the SNN has to process more temporal frames for each input sample.

Overall, the time step ablation supports the paper’s claim that temporal information is important for directly-trained SNN object detection. Our ablation experiment results reproduced the qualitative trend that higher T improves detection accuracy. However, the results also show that $T=4$ already provides most of the performance improvement, while $T=6$ gives only a slight improvement at a higher computational cost. Therefore, $T=4$ or $T=5$ appears to be a practical trade-off between accuracy and efficiency in our reproduction setup.

5.5 N-Cars Control Dataset Run

In this experiment, we lead in a new dataset to evaluate the model’s capacity to serve as a pure sequence classifier and generalise to different event-based camera data.

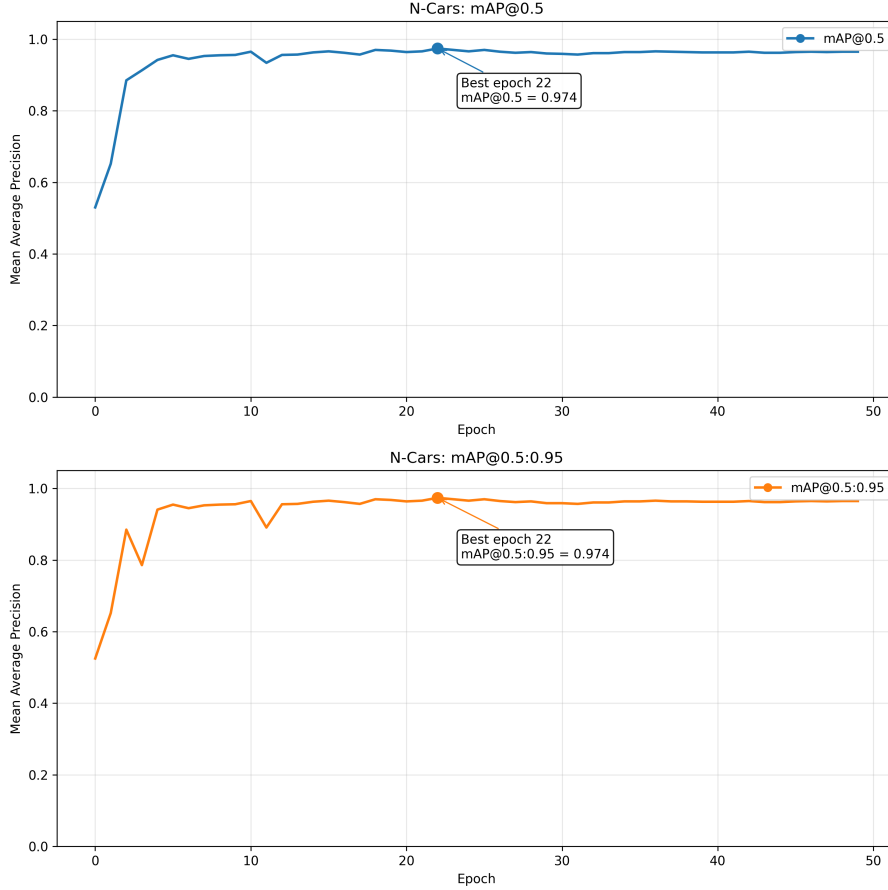


Figure 4: N-Cars control dataset results.

We adjusted the N-Cars dataset to support YOLO detection. This allowed us to keep the EMS-YOLO detection pipeline unchanged while testing whether the model can learn event-sequence-level discrimination on a different event-camera dataset.

The N-Cars control experiment completed all 50 epochs in 7.97 hours, training much faster than the Gen1 dataset runs due to the simpler nature of the dataset. It achieved very high validation performance as shown in Figure 4, with an mAP@0.5 of 0.974 and an identical mAP@0.5:0.95 of 0.975. The control run proves that the EMS-YOLO architecture easily learns to classify event streams, achieving 97.5% classification accuracy, using a ResNet-34 backbone and $T=5$ temporal steps.

Overall, the N-Cars experiment provides a successful control validation of the EMS-YOLO pipeline on a different event-based dataset. It shows that the model can be generalised to different event-based camera data.

6 Limitations

Although the reproduction experiments provide meaningful results for the main behaviour of EMS-YOLO, several practical limitations should be emphasized, as they were not discussed in detail in the results analysis.

The first major limitation is the computational constraint of the DelftBlue cluster. EMS-YOLO is expensive to train because each input sample is processed across multiple temporal steps, and increasing T directly increases the amount of computation per epoch. Most of the experiments

reached the 24-hour DelftBlue wall-time limit before the planned number of epochs was fully completed. For example, the Gen1 baseline stopped during epoch 46, and most other experiments, except the N-Cars experiment, were also terminated before completing all scheduled epochs. However, in most of these cases, the model had already converged before termination, so the main performance trends were still observable and meaningful.

The second limitation is GPU memory usage, which mainly affected the batch-size sweep. Larger batch sizes scale memory linearly, reaching 50.7 GB for BS=64, but yield only marginal accuracy gains. Attempting BS=128 on DelftBlue would exceed the A100’s 40 GB VRAM capacity, resulting in an OOM (CUDA out of memory) crash, making BS=64 the practical ceiling under our setup. The T=6 temporal-step run similarly pushed GPU memory and cache capacity close to their limits. Testing deeper backbones, broader parameter sweeps, or higher temporal resolutions would therefore require either more powerful GPUs or a multi-GPU setup.

7 Conclusion

This report reproduced EMS-YOLO, a directly-trained spiking neural network for object detection, across five experimental stages. The core claims of the paper held up: the Gen1 baseline matched and modestly exceeded the reported numbers, and the COCO2017 validation with pre-trained weights confirmed compatibility with static image inputs. The temporal step ablation on Gen1 confirmed the paper’s qualitative trend, while also suggesting T=4 as a practical efficiency sweet spot.

Beyond reproducing the paper’s experiments, a significant portion of our work involved making the codebase functional at all. Fixing GPU routing, library deprecations, tensor dimension mismatches, and a severe I/O bottleneck on the cluster’s GPFS filesystem were prerequisites to running any experiment. The N-Cars control dataset required building a classification-to-detection format conversion from scratch.

The main thing we could not do was retrain from scratch on COCO2017, which was computationally out of reach. That remains an open gap. Overall however, the architecture is sound and reproducible, and the energy-efficiency argument for directly-trained SNNs over ANN-SNN conversion methods is well-supported by what we observed.

References

- [1] Q. Su, Y. Chou, Y. Hu, J. Li, S. Mei, Z. Zhang, and G. Li, “Deep directly-trained spiking neural networks for object detection,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 6555–6565.
- [2] Y. Hu, L. Deng, Y. Wu, M. Yao, and G. Li, “Advancing spiking neural networks toward deep residual learning,” *IEEE transactions on neural networks and learning systems*, vol. 36, no. 2, pp. 2353–2367, 2024.
- [3] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, and Y. Tian, “Deep residual learning in spiking neural networks,” *Advances in neural information processing systems*, vol. 34, pp. 21 056–21 069, 2021.

A Deployment and Synchronization Scripts

A.1 Repository Synchronization Script

The local repository was synchronized with the scratch storage space on the DelftBlue cluster login node using the following bash script:

```
#!/bin/bash
# Usage: ./sync_to_delftblue.sh <netid>

set -euo pipefail

NETID="${1:-${NETID:-YOUR_NETID}}"
if [[ -z "$NETID" ]]; then
    echo "Usage: $0 [netid] (defaults to YOUR_NETID)" >&2
    exit 1
fi

REMOTE_HOST="login.delftblue.tudelft.nl"
REMOTE_PATH="~/EMS-YOLO/"

SSH_CMD="ssh -o ServerAliveInterval=60 -o ServerAliveCountMax=3"

rsync -avzh --partial --progress \
    -e "$SSH_CMD" \
    --exclude='.git/' \
    --exclude='.venv/' \
    --exclude='__pycache__/' \
    --exclude='*.pyc' \
    --exclude='.DS_Store' \
    --exclude='.mypy_cache/' \
    --exclude='.ruff_cache/' \
    --exclude='.pytest_cache/' \
    --exclude='/wandb/' \
    --exclude='slurm_logs/' \
    --exclude='logs/' \
    ./ "${NETID}@${REMOTE_HOST}:${REMOTE_PATH}"
```

A.2 DelftBlue Slurm Batch Configuration

Workloads were queued on A100 GPU nodes using the following Slurm configuration, requesting 128 GB of RAM to support RAM caching:

```
#!/bin/bash
#SBATCH --job-name=gen1-baseline
#SBATCH --account=Education-EEMCS-MSc-DSAIT
#SBATCH --partition=gpu-a100
#SBATCH --gpus-per-task=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --time=23:59:00
#SBATCH --mem-per-cpu=8000M
#SBATCH --output=slurm_logs/exp1-baseline-%j.out
```

```

module load 2024r1
module load openmpi
module load cuda/12.4
module load python

# Add virtual environment packages to Python path
export PYTHONPATH="/home/YOUR_NETID/EMS-YOLO/.venv/lib/python3.10/site-packages:$PYTHONPATH"
export WANDB_MODE=disabled

# Execute inside g1-resnet/ for event-based data run isolation
cd g1-resnet
python3 train_g1.py \
  --data data/gen1.yaml \
  --img 320 \
  --batch-size 16 \
  --epochs 50 \
  --weights "" \
  --device 0 \
  --project /scratch/$USER/runs/stage1 \
  --name gen1_baseline \
  -T 5 \
  --cache

```

B Code and Algorithm Adaptation Listings

B.1 Dynamic Tuple Length Dataloader Check

To prevent crashes resulting from variations in dataloader formats (4-element tuples for static images vs. 3-element tuples for event-frames), we implemented a length check on the yielded batch:

```

# Dynamic tuple check in val.py batch loop
if len(batch) == 4:
    im, targets, paths, shapes = batch
else:
    im, targets, paths = batch
    shapes = None

```

B.2 Static Image 4D to 5D Dimension Expansion

Static 4D image tensors ($batch, 3, H, W$) are converted to 5D ($batch, T, 3, H, W$) by unsqueezing and duplicating them along the temporal dimension T to align with the SNN backbone:

```

# Repeating static image across T steps
if is_coco:
    im = im.unsqueeze(1).repeat(1, T, 1, 1, 1)

```

B.3 Parent Process Pre-Fork ThreadPool Caching

To prevent multiprocessing worker memory duplication and subsequent OOM cgroup limits, we load and cache the dataset arrays inside the parent process during initialization using a thread pool:


```

from multiprocessing.pool import ThreadPool

# Parallel pre-caching within LoadImagesAndLabels.__init__ in datasets_g1T.py
results = ThreadPool(8).imap(
    lambda x: self.load_image_and_label(x),
    range(self.n)
)
for i, (out_img, labels) in enumerate(results):
    self.ims[i] = out_img
    self.labels[i] = labels

```

B.4 Gen1 H5 Sequence Read Optimization

To speed up the slicing of raw Gen1 H5 containers on DelftBlue’s GPFS filesystem, we loaded the entire event array into RAM at the sequence start instead of querying the H5 file multiple times per timestamp:

```

import h5py

with h5py.File(h5_path, 'r') as h5_file:
    # Read the full dataset into RAM once to avoid GPFS random disk I/O bottlenecks
    events = h5_file['events'][:]

```

B.5 N-Cars Classification-to-Detection Mapping

N-Cars classification sequences were mapped to detection format by writing full-frame bounding boxes for positive samples and generating empty label files for backgrounds:

```

# Mapping positive samples to YOLO coordinates in get_ncars_data.py
# Format: <class_id> <x_center> <y_center> <width> <height>
yolo_label = "0 0.5 0.5 1.0 1.0\n"
with open(out_label_path, 'w') as f:
    f.write(yolo_label)

```

B.6 Pillow 10+ Text Measurement Fallback

To handle the deprecation and removal of ‘.getsize()’ in modern Pillow builds on DelftBlue compute nodes, we implemented a bounding-box calculation fallback:

```

# Bounding-box text size calculation fallback in utils/plots.py
try:
    w, h = self.font.getsize(text)
except AttributeError:
    left, top, right, bottom = self.font.getbbox(text)
    w = right - left
    h = bottom - top

```

C Results Elaborated

C.1 Baseline Reproduction(Gen1 dataset)

Method	Model	Params	T	Firing Rate	mAP @0.5:0.95	mAP @0.5
Sew-Reset	VGG-11+SSD	12.64M	5	22.22%	0.174	/
	MobileNet-64+SSD	24.26M	5	29.44%	0.147	/
	DenseNet121-24+SSD	8.20M	5	37.20%	0.189	/
ANN	ResNet10		/	/	0.247	0.504
EMS-ResNet	EMS-Res10	6.20M	5	21.15%	0.267	0.547

Table 2. Results on Gen1 Dataset.

Figure 5: Paper’s results on Gen1 Dataset[1]

Reproduction Results:

Table 6: Experiment 1: Baseline

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.682	0.555	0.596	0.315
10	0.699	0.583	0.618	0.337
15	0.720	0.583	0.623	0.342
20	0.725	0.578	0.620	0.343
25	0.731	0.575	0.615	0.344
30	0.715	0.583	0.614	0.342
35	0.738	0.564	0.610	0.343
40	0.738	0.562	0.605	0.343
45	0.742	0.557	0.607	0.345

C.2 COCO2017 Dataset Validation

Method	Work	Model	Time Step(T)	mAP@0.5	
Backpropagation	ANN	Tiny-Yolo	/	0.262	
		ResNet50	/	0.460	
		ResNet34	/	0.565	
ANN-SNN	Spiking-Yolo [23]		3500	0.257	
	Bayesian Optimization [22]	Tiny-Yolo	500	0.211	
			5000	0.258	
	Conversion	Spike Calibration [26]	ResNet50+Burst+ MLIPooling+SpiCalib	64	0.331
				128	0.436
256				0.453	
512				0.454	
Directly-Trained SNN	EMS-YOLO	EMS-ResNet34	4	0.501	

Table 1. Results on the COCO2017 DataSet.

Figure 6: Original paper results on the COCO2017 dataset.[1]

C.3 Hyperparameter Sweep Experiment

C.3.1 Learning Rate Sweep Experiment

Table 7: Learning rate = 0.001

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.623	0.450	0.488	0.223
10	0.650	0.504	0.538	0.264
15	0.654	0.521	0.549	0.274
20	0.666	0.522	0.555	0.281
25	0.678	0.526	0.567	0.288
30	0.682	0.525	0.565	0.290
35	0.665	0.531	0.562	0.291
40	0.680	0.525	0.562	0.291
45	0.680	0.514	0.560	0.288

Table 8: Learning rate = 0.1

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.689	0.549	0.589	0.306
10	0.568	0.536	0.545	0.269
15	0.523	0.457	0.464	0.230
20	0.523	0.387	0.393	0.197
25	0.638	0.390	0.414	0.205
30	0.614	0.368	0.388	0.190
35	0.639	0.415	0.439	0.215
40	0.671	0.467	0.497	0.255

C.3.2 Batch Sweep Experiment

Table 9: Batch sweep for size = 8

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.665	0.567	0.6	0.318
10	0.724	0.574	0.623	0.34
15	0.716	0.581	0.62	0.342
20	0.719	0.585	0.623	0.345
25	0.74	0.571	0.62	0.344
30	0.726	0.581	0.616	0.346
35	0.739	0.576	0.617	0.347
40	0.742	0.571	0.611	0.346

Table 10: Batch sweep for size = 16

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.68	0.556	0.597	0.317
10	0.715	0.578	0.623	0.339
15	0.707	0.589	0.621	0.345
20	0.739	0.573	0.621	0.345
25	0.721	0.583	0.619	0.345
30	0.74	0.575	0.614	0.342
35	0.742	0.566	0.609	0.344
40	0.723	0.578	0.612	0.346
45	0.729	0.562	0.606	0.345

Table 11: Batch sweep for size = 32

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.688	0.547	0.592	0.311
10	0.709	0.575	0.615	0.336
15	0.726	0.578	0.621	0.341
20	0.734	0.582	0.622	0.342
25	0.738	0.573	0.617	0.341
30	0.731	0.575	0.614	0.343
35	0.753	0.558	0.614	0.343
40	0.734	0.562	0.61	0.342
45	0.729	0.562	0.609	0.343

Table 12: Batch sweep for size = 64

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.685	0.537	0.586	0.303
10	0.697	0.58	0.616	0.339
15	0.731	0.572	0.617	0.341
20	0.726	0.575	0.617	0.342
25	0.702	0.582	0.611	0.342
30	0.737	0.566	0.613	0.343
35	0.724	0.569	0.609	0.343
40	0.756	0.551	0.608	0.344
45	0.744	0.548	0.605	0.343

C.4 Ncars experiment

Table 13: N cars experiment

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.896	0.896	0.955	0.955
10	0.887	0.944	0.965	0.965
15	0.890	0.932	0.966	0.966
20	0.900	0.915	0.964	0.964
25	0.930	0.910	0.970	0.970
30	0.898	0.914	0.959	0.959
35	0.923	0.910	0.964	0.964
40	0.917	0.925	0.963	0.963
45	0.925	0.917	0.964	0.964

C.5 Temporal Time Step Ablation study

T	1	3	5	7
mAP@0.5	0.328	0.362	0.367	0.383
mAP@0.5:0.95	0.162	0.184	0.189	0.199

Table 5. Impact of different size of time steps on COCO Dataset.

Figure 7: Paper timestep ablation results on the COCO2017 dataset. [1].

Reproduction Results

Table 14: Timestep = 1 experiment

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.656	0.487	0.531	0.269
10	0.655	0.524	0.559	0.294
15	0.661	0.531	0.563	0.298
20	0.659	0.539	0.567	0.302
25	0.704	0.523	0.569	0.305
30	0.687	0.533	0.565	0.305

Table 15: Timestep = 2 experiment

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.650	0.532	0.568	0.295
10	0.678	0.557	0.589	0.317
15	0.679	0.565	0.591	0.322
20	0.717	0.549	0.592	0.324
25	0.724	0.550	0.592	0.324
30	0.737	0.538	0.588	0.328
35	0.718	0.547	0.589	0.329
40	0.714	0.550	0.585	0.329
45	0.713	0.537	0.578	0.328

Table 16: Timestep = 4 experiment

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.681	0.556	0.596	0.316
10	0.725	0.564	0.613	0.336
15	0.724	0.573	0.617	0.340
20	0.718	0.579	0.617	0.342
25	0.730	0.573	0.614	0.342
30	0.735	0.563	0.612	0.341
35	0.725	0.567	0.605	0.343
40	0.711	0.571	0.607	0.345
45	0.754	0.545	0.602	0.343

Table 17: Timestep = 6 experiment

Epoch	Precision	Recall	mAP@0.5	mAP@0.5:0.95
5	0.674	0.562	0.595	0.316
10	0.726	0.576	0.621	0.342
15	0.720	0.584	0.624	0.345
20	0.719	0.585	0.620	0.346
25	0.724	0.582	0.616	0.346
30	0.735	0.574	0.613	0.345
35	0.742	0.566	0.610	0.345

D Contributions

Table 19 summarizes the primary responsibilities of each team member. While individual experiments were assigned to specific members, all members participated in discussions, presentations, and report writing.

Table 18: Distribution of Work

Name	Contribution
Eugen	Developed the ingestion pipeline, preprocessing the datasets. Conducted the baseline reproductions (Gen1, COCO) and the new dataset experiment (N-cars).
Senquan	Developed the dynamic temporal resampling dataloader extensions and the execution scripts for the temporal timestep ablation studies ($T \in \{1, 2, 4, 6\}$).
Adhiraj	Developed the parameter sensitivity sweep infrastructure, managing parallel runs for batch sizes ($BS \in \{8, 16, 32, 64\}$) and learning rates ($LR \in \{0.001, 0.1\}$).
All Members	Participated in research formulation, design discussions, experimental result interpretation, visual presentation preparation, and the collaborative writing and review of the final report.

Table 19: Division of work within the project.

Name	Contribution
Eugen	Developed the full DelftBlue deployment pipeline including cluster configuration, storage migration, and Slurm job scripts. Led all codebase adaptations (Phases 1-4): environment compatibility fixes, GPFS I/O optimization, pre-fork RAM caching, and N-Cars classification-to-detection mapping. Conducted the Gen1 baseline , COCO2017 validation , and N-Cars experiments.
Senquan	Developed the temporal step ablation experiment scripts and conducted training runs across $T \in \{1, 2, 4, 6\}$. Contributed to results analysis and report writing for Sections 5.4 and Appendix D.
Adhiraj	Developed the hyperparameter sensitivity sweep scripts and conducted training runs across batch size and learning rate configurations. Contributed to results analysis and report writing for Section 5.3 and Appendix C.3.
All Members	Participated in project planning, implementation discussions, result interpretation, and report writing.